

Guess Market – Exercise 3 (Client / Server)

Implemented bonus: Chat (5 points). All logged-in users share one chat room. The chat is a tab in the client, is refreshed by pull every 0.5 seconds and only the new lines are sent each time (by version number).

Submitters

Name	ID	Email
Shady Aboarya	325556835	shadyaburaya04@gmail.com
Abed Alqader Wattad	213726094	shadyaburaya04@gmail.com

GitHub repository: <https://github.com/Shady-ab/GuessMarket-EX3>

Submission content and how to run

Item	Description
GuessMarket.war	The server. Copy it into tomcat/webapps; Tomcat deploys it under the context path /GuessMarket. The WAR contains everything it needs in WEB-INF/lib (engine jar, DTO jar, Gson).
GuessMarketClient\	The JavaFX client: GuessMarketClient.jar, GuessMarketDto.jar, gson-2.11.0.jar, the JavaFX jars in lib\ and run.bat. Double-click run.bat to start a client. Several clients can run at the same time.
samples\	The course sample files and schema (samples\course) and our own example files (valid files and files with errors).

The client connects to `http://localhost:8080/GuessMarket` automatically. For a different server one can run `run.bat http://host:port/GuessMarket` (optional). Requirements: Java 21 or newer, Tomcat 10.1 (Jakarta Servlet 6).

Typical check: start Tomcat with the WAR, run two clients, log in with two different names, load `samples\ex3-valid-binary.xml` from the first client, deposit money, open an event and trade from the second client.

Design decisions

Users and login

- Login is by user name only – no passwords and no sign-up, as required. Names are trimmed, must not be empty and are at most 40 characters. Any language (including Hebrew) is allowed.
- A name that belongs to a user who is **currently online** is rejected (HTTP 409) with a clear message, and the login screen lets the user try another name.

- A user is "online" while the client keeps polling (last request less than 6 seconds ago). After Logout, closing the window or 6 seconds without requests, the same name can log in again and **continues the same account** (balance, holdings, ledger). This is the natural behaviour for a system without passwords and without persistence.
- A new user starts with balance 0 and adds money with **Deposit** (positive amount, at most 1,000,000,000).
- If an action makes the balance negative (for example the market maker covers a deficit when closing an event) the user is **blocked** from trading and opening events. A deposit that brings the balance back to 0 or more unblocks the user automatically.

Events and XML upload

- The XML is uploaded as multipart/form-data and parsed directly from the request stream in memory. **The file content is never saved on the server**: the servlet's `fileSizeThreshold` equals the maximum file size (5 MB), so Tomcat never writes a temporary file, and nothing is kept after parsing. Apache Commons is not used.
- Validation: everything from EX1 (structure, description, commission $0 \leq 90$ with type on-close/on-purchase, exactly one market method, LMSR $b > 0$, order book $d > 0$, initial investment ≥ 0 , allow-mint true/false), at least two options with unique names (case-insensitive), no duplicate event names inside the file, and **no event whose name already exists on the server**. A file with any error is rejected as a whole and nothing is added.
- Events from all uploads accumulate. Each event gets a serial id from the server (the XML has no event id). The uploading user becomes the market maker of all events in the file.
- The spelling variants `comision` and `inita1` that appear in the course examples are accepted together with `commission` and `initial`.
- **Multi-option events** (more than 2 options) are fully supported:
 - LMSR: cost $C(q) = b \cdot \ln(\sum e^{q_i/b})$ (computed with log-sum-exp for stability), price of option $i = \text{softmax}$, and the market maker pays the subsidy $b \cdot \ln(N)$ when opening.
 - Order book: one book per option with price-time priority; a trade is executed at the resting order's price. With allow-mint, if the incoming bid plus the best bids of all other options reach d , a complete set is minted: the resting bids pay their price and the incoming bid pays the complement $\max(0, d - \text{sum})$. The initial investment buys $\text{initial} / d$ complete sets for the market maker.
- Commissions: on-purchase is paid by the buyer on top of the price and goes to the market maker immediately; on-close is taken from the winners' payout when the event is closed. Whatever remains in the event account after closing is returned to the market maker. Every money movement appears as a separate ledger line.

Server

- One engine instance is stored in the `ServletContext`. All engine methods are synchronized, so concurrent requests are safe and the JSON snapshots sent to clients are consistent. XML parsing happens before taking the lock, so a large upload does not block other users.
- All responses are JSON (Gson). Errors are returned as `{"error": "..."} with a proper status code (400 invalid input, 401 not logged in, 409 name taken, 500 unexpected), and the client shows the message as is.`
- The logged-in user is kept in the HTTP session (cookie `JSESSIONID`). No data is persisted – restarting Tomcat starts from scratch, as required.

Client

- HTTP is done with the JDK `java.net.http.HttpClient` (asynchronous, with a cookie manager), so the client needs no extra libraries except Gson and JavaFX. Network calls never run on the JavaFX thread.

- Refresh by pull every 0.5 seconds. Lists that are small (events, users, selected event) are fetched whole and the screen is updated only if the content changed; growing lists (ledger, chat) are fetched as deltas (ledgerFrom / version). A poll is skipped while the previous one is still running.
- If the server stops or the session expires, the client returns to the login screen with an explanation.
- The window size is computed from the screen size, and every screen uses resizable layouts (split panes, growing tables). The EX2 components were reused: filters, event table, event details text, trade dialogs and formatting.

Screens

- **Login** – user name only; errors (name taken, server not running) are shown on the same screen.
- **Events** – laid out as in the course sketch. On the left: the filter line (method, status, commission, "only my MM events") and all events in the system. On the right, "Event details and trade": the event's details, the actions Participate / Trade, Open and Close (the last two for the market maker only), one box per option (its order book with bids and asks for order-book events, or its LMSR price), and the participations information and trade history. "Load events XML" is in the top bar.
- **My account** – name, balance, blocked state, Deposit; my events with my involvement (as in EX2); the ledger, one line per action including commissions received; and all other users with balance and whether they are market makers.
- **Chat** (bonus).

Modules and main classes

The project has four modules: **GuessMarketEngine** (logic, updated from EX2), **GuessMarketDto** (JSON objects shared by server and client), **GuessMarketServer** (servlets, packed into the WAR) and **GuessMarketClient** (new JavaFX client).

Engine (`guessmarket.engine`)

Class	Role
GuessMarketEngine / GuessMarketEngineImpl	Multi-user engine: login/logout and online tracking, adding events from an XML stream, deposit, open, buy (LMSR), place order, close. Thread-safe.
EventXmlParser, EventDefinition	New EX3 parser that reads the XML from an InputStream (no file), validates it and returns event definitions.
MarketEvent	One event with N options: LMSR pricing, order books, matching, minting, commissions and closing.
User	Balance, holdings, blocked flag, market-maker events, online time and the ledger. Every credit/debit adds a ledger line.
LedgerEntry	One ledger line: sequence number, time, description, amount, balance after.
OptionOrderBook, BookOrder, TradeRecord	Order book of one option, a resting order and a trade history record (from EX2, extended for N options).

DTO (`guessmarket.dto`)

Class	Role
Api	Context path, endpoint paths and parameter names – one place shared by server and client.
EventSummaryDto, EventDetailsDto, OptionDto, BookDto, OrderDto, TradeDto, ParticipantDto	Events list and full event details.
MyUserDto, HoldingDto, LedgerLineDto, UserSummaryDto	My account (balance, events, ledger delta) and the other users list.
ChatDto, ChatLineDto, MessageDto	Chat delta, and the success/error message of every action.

Server (guessmarket.server)

Class	Role
utils.ApiServlet	Base class of all servlets: UTF-8, JSON, no-cache, login check, updates the user's online time, converts exceptions to JSON errors with status codes.
utils.ServletUtils, utils.SessionUtils	Shared engine / chat manager in the ServletContext, Gson instance, user name in the session.
DtoMapper	Builds DTOs from engine objects under the engine lock (consistent snapshot).
chat.ChatManager	Chat lines with a version number for delta requests (bonus).
servlets.*	One servlet per action, listed below.

Endpoint	Method	Purpose
/login, /logout	POST	Log in by name / log out.
/upload	POST multipart	Upload an events XML (part file).
/events	GET	All events (summary).
/event?id=	GET	Details of one event.
/event/open, /event/close	POST	Open / close an event (market maker only).
/event/buy, /event/order	POST	Buy LMSR shares / place a buy or sell order.
/me?ledgerFrom=	GET	My account and new ledger lines.
/users	GET	All users: name, balance, market maker, online.

/deposit	POST	Add money.
/chat?version=	GET / POST	Read new chat lines / send a line (bonus).

Client (`guessmarket.client`)

Class	Role
GuessMarketClientApp	Main class; one window that switches between the login screen and the main screen; logs out on close.
api.ServerApi	All HTTP calls (GET JSON, POST form, multipart upload) with the session cookie; turns error responses into readable messages.
api.Poller	Runs the refresh tasks every 0.5 seconds in the background without overlapping requests.
ui.LoginView, ui.MainView	Login screen; main screen with top bar (user, balance, load XML, logout) and tabs.
ui.EventsPane	Events screen: filter line and events table (reused from EX2) next to the details panel.
ui.EventDetailsView	"Event details and trade" panel: event details, actions, a box per option with its order book or LMSR price, participations and trade history.
ui.UserPane	My account screen: details, deposit, my events, ledger and other users.
ui.ChatPane	Chat (bonus).
ui.TradeDialogs, ui.EventText, ui.UiFormat	EX2 dialogs and text formatting adapted for N options.
ui.ClientSession	Shared state of the logged-in client and the common way to run an action and show its result.

Sample files

File	Expected result
course\ex3-course-three-events.xml	Course file: loads 3 events (2 order-book, 1 LMSR). Loading it a second time is rejected because the names already exist.
course\ex3-course-mujtaba.xml	Course file: loads 1 LMSR event; the events of both files accumulate.
ex3-valid-binary.xml	Loads binary LMSR and order-book events.

ex3-valid-multi-option.xml	Loads a 4-option LMSR event and a 3-option order-book event with mint.
ex3-error-commission.xml	Rejected: commission out of range.
ex3-error-duplicate-name.xml	Rejected: two events with the same name in the file.
ex3-error-existing-name.xml	Rejected after ex3-valid-binary.xml was loaded: event name already exists.
ex3-error-one-option.xml	Rejected: an event with fewer than two options.